

Getting the Wrong Identity in Microsoft SQL Server identity Columns?

By Don Schlichting

This article will explore Microsoft SQL server identity columns, including their problems, use and scope.

Introduction

An Identity column is used in SQL server to create a surrogate key value for a table. This will be a unique identifier usually in sequential order. Starting at some predefined number, the Identity column increments every time a new record is added to the table. For MS Access users, this is comparable to an Auto Numbering field. For Oracle users, the Identity column can be thought of as a sequence built into a table.

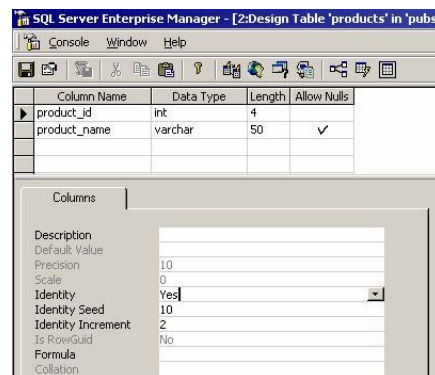
Creating

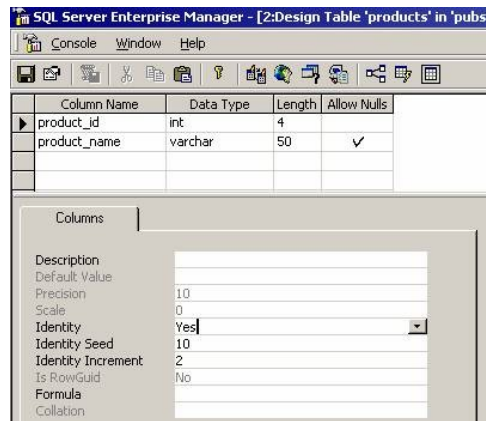
The Identity key word is supported in both the create and alter table statements. The following statement will create a new table with the product_id column as an identity field.

```
USE pubs
IF EXISTS(SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_NAME = 'products')
DROP TABLE products
GO
CREATE TABLE products
(
product_id int IDENTITY(10,2),
product_name varchar(50)
```

The first number in the Identity function is for the seed value. The seed will be the first number used as an identity. In our case, the first value in the table will be 10. The second number is the increment. The products table id will count up by twos.

To create an identity field from Enterprise Manager, set the Identity to Yes, and enter a seed and increment. A seed of 1 with an increment of 1 is the default.





Inserting

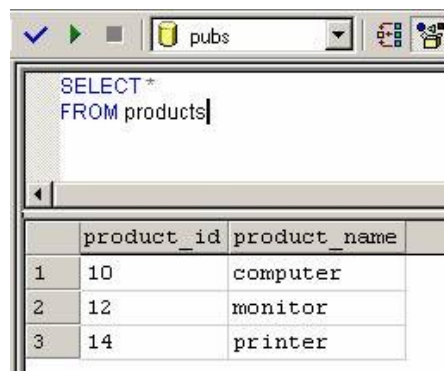
Enter three product names into the new table using an insert into statement. We will only enter the product_name, letting sql create the identity id.

```
INSERT INTO products
    (product_name)
VALUES
    ('computer')
```

```
INSERT INTO products
    (product_name)
VALUES
    ('monitor')
```

```
INSERT INTO products
    (product_name)
VALUES
    ('printer')
```

Selecting the new rows out will show our first seed of 10. Additional product names will increment by two.



By default, values cannot be inserted into an identity field. The statement:

```
INSERT INTO products
```

```
        (product_id, product_name)
VALUES
    (18, 'printer')
```

Will fail with error:

Server: Msg 544, Level 16, State 1, Line 1
Cannot insert explicit value for identity column in
table 'products' when IDENTITY_INSERT is set to OFF.

To force our value, the IDENTITY_INSERT needs to be set ON:

```
SET IDENTITY_INSERT products ON
```

```
INSERT INTO products
    (product_id, product_name)
VALUES
    (18, 'printer')
```

```
SELECT *
FROM products
```

	product_id	product_name
1	10	computer
2	12	monitor
3	14	printer
4	18	printer

The new printer with an id of 18 has been successfully entered.

We now have a gap in our numbering, from identity 14 to 18. This will not create any problems for SQL. The next identity used will be 20. However, if gaps will cause a problem for your particular application, search BOL for "Use generic syntax for finding gaps in identity values" for detailed examples of how to find gaps.

Getting the Wrong Identity in Microsoft SQL Server identity Columns? - Page 2

By Don Schlichting

Discovery and Maintenance

There are a few functions to help discover the status of the identity field.

SELECT IDENT_SEED('products') ,will report 10 as our seed number.

SELECT IDENT_INCR('products') ,displays 2 as the increment.

SELECT MAX(IDENTITYCOL) FROM products, reports 18 as the highest identity used.

DBCC CHECKIDENT checks the validity of the identity field, and corrects it if need be, or changes the seed value.

The statement:

```
DBCC CHECKIDENT (products, NORESEED)
```

Returns:

Checking identity information: current identity value '18', current column value '18'.

DBCC execution completed. If DBCC printed error messages, contact your system administrator.

The CHECKIDENT statement can accept up to 3 parameters: the table name, reseed, and the optional new reseed number. With NORESEED set, a check will be done without any corrective actions. If there were errors to correct, a value of RESEED would be used. Another benefit of the CHECKIDENT statement is to skip ahead in the identity numbering. If for some reason, we needed new inserts to now begin at 52, the statement and return would be:

```
DBCC CHECKIDENT (products, RESEED, 50)
```

Checking identity information: current identity value '18', current column value '50'.
DBCC execution completed. If DBCC printed error messages, contact your system administrator.

The next item added would have an identity of 52, the new seed of 50 plus the increment of 2.

DBCC CHECKIDENT (products, RESEED, 50)		
INSERT INTO products		
(product_name)		
VALUES		
('cdrom')		
	product_id	product_name
1	10	computer
2	12	monitor
3	14	printer
4	18	printer
5	52	cdrom

Getting the Identity

If you have been working with other databases, the methods for getting SQL identity values may not seem straightforward. In some databases, your auto number field is assigned at the beginning of the insert. In Oracle, if you are using a sequence, you have the identity before you start the insert. With SQL, the new identity is not known until after the insert completes. This leads to problems when you need that new identity to continue with a different transaction, or return the value to your application for future use.

There are three methods for getting the new identity. The difference between them is the scope and session they report on. Use the incorrect one, and the result back may be the wrong identity.

SCOPE_IDENTITY

The function SCOPE_IDENTITY returns the new identity created on any table in this session in this scope, giving us the new value we would expect. In this case, a new product will be entered that should have the next identity of 54.

```
INSERT INTO products
    (product_name)
VALUES
    ('dvd')

SELECT SCOPE_IDENTITY()
```

	(No column name)
1	54

Just to confirm:

```
SELECT * FROM Products where product_name = 'dvd'
```

	product_id	product_name
1	54	dvd

@@IDENTITY

This global variable is usually the first one suggested in books and news groups for discovering new identities. However, its real purpose is not at first evident and may produce the wrong identity. @@IDENTITY will return the last identity used in your session, but across all scopes. Meaning if your statement causes a trigger to fire, and that trigger insert into a table with an identity field, the trigger's new identity will be returned, not your statement's new identity. For example, create a test table with an identity, and a trigger on the products table that will insert on the test table.

```
CREATE TABLE test(id int IDENTITY(100,1))
GO
CREATE TRIGGER trigger1 ON products FOR INSERT
AS
BEGIN
    INSERT test DEFAULT VALUES
END
GO
```

Now when an insert on products is done, the trigger will cause an insert on test. After, we will get the new identities using both methods discussed.

```
INSERT INTO products
    (product_name)
VALUES
    ('speakers')
```

```
SELECT @@IDENTITY
--returns 100, the value from the test identity caused by the trigger
```

```
SELECT SCOPE_IDENTITY()
--returns 56, from our statement in the products table
```

The different returns are due to the difference in scopes examined by each. If you want your identity, use SCOPE_IDENTITY(). If the identity wanted is for any triggers underneath, use @@IDENTITY.

The third method uses the IDENT_CURRENT function. IDENT_CURRENT requires a parameter for the table name. Like @@IDENTITY, it will look at any scope, but in addition, it will look any session, not just your own.

Conclusion

Using SQL identity fields provides a quick and effective way of generating unique values. If the correct method is used for retrieving new identities, your application will be assured in not getting the Wrong Identity.